



6



8



960 sp

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

تمارين عن lista

عملي مشترك



RB Informatics; 7/06/2026

محتوى مجاني غير مخصص للبيع التجاري

مبادئ الذكاء الصناعي

تمرين: لدينا الحقائق التالية التي تعبر عن صلة الأبوة حيث $child(X, Y)$ تعني أن Y هو ابن X

martha ابن charlotte

charlotte ابن caroline

caroline ابن laura

laura ابن rose

المطلوب: اكتب Predicate باسم $descende(X, Y)$ تتج إذا كان Y حفيداً أو من سلالة الشخص X

الحل:

```

main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
child(martha, charlotte).
child(charlotte, caroline).
child(caroline, laura).
child(laura, rose).
descende(X, Y) :- child(X, Y).
descende(X, Y) :- child(X, Z), descende(Z, Y).

```

Colourising buffer ... done, 0.02 seconds, 21 fragments Line: 7

الكود مبني على مفهوم الاستدعاء الذاتي (Recursion) ليتتبع شجرة العائلة من الأعلى إلى الأسفل:

الحالة الأساسية: (Base Case)

$descende(X, Y) :- child(X, Y)$

أي: يكون Y من سلالة X مباشرة إذا كان Y هو الابن المباشر لـ X مثل (مارثا وشارلوت)

الحالة العودية (Recursive Case): $descende(X, Y) :- child(X, Z), descende(Z, Y)$

أي إذا لم يكن Y ابناً مباشراً لـ X يبحث البرنامج عن وسيط Z أي يكون Y من سلالة X إذا كان لـ X ابن مباشر اسمه

Z ، وهذا الـ Z نفسه يكون جداً أو أصلاً لـ Y (أي Y ينحدر من Z)



↪ إذا استدعينا. :descende(martha, caroline).

- سيفحص الشرط الأول: هل كارولين ابن مباشر لمارثا؟ لا
 - سينتقل للشرط الثاني: يبحث عن ابن مباشر لمارثا، فيجد $Z = charlotte$
 - الآن يستدعي التابع نفسه مجدداً ليفحص العلاقة بين الابن والهدف. :descende(charlotte, caroline).
 - هنا يتحقق الشرط الأول مباشرةً لأن كارولين ابن لشارلوت، فتعود النتيجة true
- عند الاستدعاء:

```

SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
?- descende(martha, charlotte).
true
Unknown action: A (h for help)
Action?

```

مثال: اكتب Predicate باسم `get_num(From, To, R)` يقوم بإرجاع جميع الأعداد الصحيحة المحصورة ضمن مجال معين يبدأ من القيمة `From` وينتهي عند القيمة `To` بالتتالي.

الحل:

```

main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
get_num(To, To, To) :- !.
get_num(From, To, R) :-
    From < To,
    R = From.
get_num(From, To, R) :-
    Fl is From + 1,
    get_num(Fl, To, R).

```

Buffer saved in file 'main.pl' Line: 8

↪ شرط التوقف النهائي(Base Case):

`get_num(To, To, To):- !.`

- أي: عندما تزيد القيمة الأولى وتتساوى تماماً مع قيمة النهاية (To) يرجع البرنامج هذه القيمة الأخيرة كحل ويتوقف تماماً عن البحث بسبب وجود الـ Cut
- إرجاع الرقم الحالي: `get_num(From, To, R) :- From < To, R = From.`
- أي: إذا كان الرقم الحالي `From` أصغر من النهاية `To`، يقوم البرنامج فوراً بمساواة المتغير الناتج `R` بالقيمة الحالية ويعيدها كحل أول (مثلاً يعيد الرقم 1).

↪ الانتقال للرقم التالي (Recursive Step) :

- `get_num(From, To, R) :- F1 is From + 1, get_num(F1, To, R)`.
- أي: عندما نطلب من البرنامج الحل التالي (بالضغط على ; ينتقل لهذه القاعدة أي يقوم بزيادة الرقم الحالي بمقدار 1 ليدخل في متغير جديد F1، ثم يستدعي الدالة نفسها من جديد لتكرار العملية مع الرقم التالي (2، ثم 3، وهكذا) حتى يصل لشرط التوقف.

دعنا نأخذ مثالاً بسيطاً لتتبع الاستدعاء التالي: `get_num(1, 3, R)`. أي نريد طباعة الأرقام من 1 إلى 3

↪ الخطوة 1: المحاولة الأولى (توليد الرقم 1)

يدخل البرنامج بالقاعدة الأولى `get_num(1, 3, R)`. هل $1 = 3$ ؟ لا، يفشل شرط التوقف.

ينتقل للإستراتيجية الثانية:

يتحقق من الشرط $1 < 3$ ؟ نعم.

ينفذ $R = 1$ أي $R = \text{From}$

النتيجة الأولى: يطبع البرنامج $R = 1$ ويتوقف مؤقتاً منتظراً ضغط ; من المستخدم للبحث عن حل آخر.

↪ الخطوة 2: المحاولة الثانية بعد ضغط ; (توليد الرقم 2)

بما أن المستخدم طلب حلاً آخر، يعود البرنامج ويذهب إلى القاعدة الثالثة (الاستدعاء العودي) من مكان ما توقف عند الرقم 1:

يحسب $F1 = 1 + 1$ تصبح $F1 = 2$

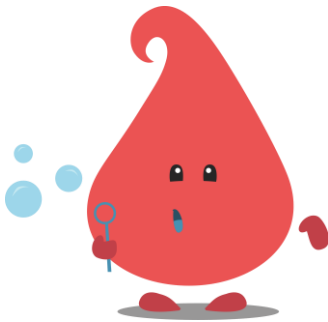
يستدعي البرنامج نفسه مجدداً `get_num(2, 3, R)`

في هذا الاستدعاء الجديد:

○ القاعدة الأولى تفشل لأن $2 \neq 3$

○ القاعدة الثانية تنجح لأن $2 < 3$ ، ويقوم بإسناد $R = 2$

○ النتيجة الثانية: يطبع البرنامج $R = 2$ ويتوقف منتظراً ضغط ;



↪ الخطوة 3: المحاولة الثالثة بعد ضغط ; (توليد الرقم 3)

يطلب المستخدم حلاً آخر، فينتقل البرنامج للقاعدة الثالثة للاستدعاء: `get_num(2, 3, R)`

يحسب $F1 = 2 + 1$ تصبح $F1 = 3$

يستدعي البرنامج نفسه مجدداً `get_num(3, 3, R)`

في هذا الاستدعاء الجديد:

يدخل القاعدة الأولى `get_num(3, 3, R)`

هنا تطابقت المتغيرات تماماً (To, To, To) أي أصبح الاستدعاء `get_num(3, 3, 3)`، وبالتالي تصبح قيمة $R = 3$

النتيجة الثالثة: يطبع البرنامج 3 = R

بسبب وجود الـ Cut في القاعدة الأولى، عندما يضغط المستخدم؛ مجدداً، لن يجد البرولوج أي مسارات بديلة مخزنة في الذاكرة للرجوع إليها، فيتوقف البرنامج تماماً ويظهر false أو ينتهي التنفيذ.

○ وعند الاستدعاء:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help

?- get_num(1, 3, X).
X = 1 ,
X = 2 ,
X = 3 .
?-
```

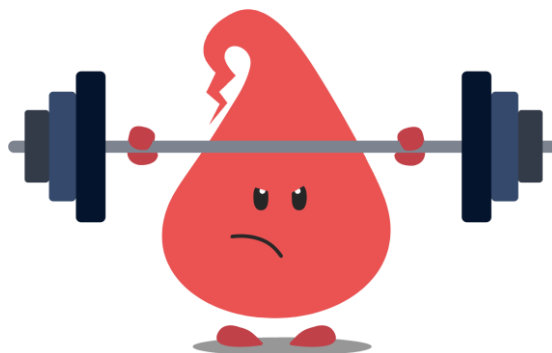
مثال: لدينا الحقائق التالية التي تمثل الرحلات الجوية المباشرة بين مجموعة من المطارات:

- رحلة مباشرة من لندن إلى باريس.
- رحلة مباشرة من باريس إلى روما.
- رحلة مباشرة من روما إلى أثينا.
- رحلة مباشرة من لندن إلى برلين.
- رحلة مباشرة من برلين إلى وارسو.
- رحلة مباشرة من مدريد إلى لشبونة.

المطلوب:

1. قم بتعريف قاعدة باسم `connected(A, B)` تسمح بالطيران في كلا الاتجاهين) بمعنى إذا كان هناك طيران مباشر من المطار A إلى المطار B ، فيجب أن يعتبر البرنامج تلقائياً وجود طيران من B إلى A أيضاً
2. اكتب قاعدة عودية (Recursive Predicate) باسم `travelFromTo/2` تُعطي True إذا كان من الممكن السفر من المطار X إلى المطار Y ، سواء كان الطيران مباشراً أو يتضمن محطات توقف
3. اكتب قاعدة باسم `route/3` تقوم بتوليد قائمة تحتوي على جميع المطارات التي يمر بها المسافر بالترتيب أثناء رحلته من مطار البداية إلى مطار النهاية.

ملاحظة: يمكننا استعمال fail, cut عند اللزوم



```

main.pl [modified]
File Edit Browse Compile Prolog Pce Help
main.pl [modified]
directFlight(london, paris).
directFlight(Paris, rome).
directFlight(rome, athens).
directFlight(london, berlin).
directFlight(berlin, warsaw).
directFlight(madrid, lisbon).
connected(X, Y) :- directFlight(X, Y).
connected(X, Y) :- directFlight(Y, X).
has_visited(X, [X|_]) :- !, fail.
has_visited(X, [_|T]) :- has_visited(X, T).
route(X, Y, Route) :-
    path(X, Y, 5, [X], ReverseRoute),
    reverse(ReverseRoute, Route).
path(Y, Y, _, Visited, Visited) :- !.
path(X, Y, N, Visited, Route) :-
    N > 0,
    connected(X, Z),
    \+ has_visited(Z, Visited),
    N1 is N - 1,
    path(Z, Y, N1, [Z|Visited], Route).
travelFromTo(X, Y) :- route(X, Y, _), !.

```

أولاً:

قمنا بتعريف الحقائق ثم فكرنا بما أن الطيران مسموح بالاتجاهين، يجب وضع شروط حتى لا يظل البرنامج يتنقل بين (لندن الى باريس ثم لندن ثم باريس) إلى الأبد دون أن يصل للهدف ومن اجل السماح بالطيران العكسي الحقائق المكتوبة في الأعلى تعبر عن اتجاه واحد فقط (مثلاً من لندن لباريس). بفضل هاتين القاعدتين، أصبح البرنامج يفهم تلقائياً أنه إذا وُجد طريق من X إلى Y فبالأكيد يمكن العودة من Y إلى X

`.connected(X, Y) :- directFlight(X, Y)`

`.connected(X, Y) :- directFlight(Y, X)`

ثانياً:

تعريف قائمة المطارات المزاراة (has_visited) لمنع اللوب:

`.has_visited(X, [X|_]) :- !, fail`

`.has_visited(X, [_|T]) :- has_visited(X, T)`

هذه دالة مساعدة تبحث داخل قائمة المطارات التي زرتها بالفعل أثناء الرحلة الحالية (Visited)

آلية عملها: إذا كان المطار X موجوداً في رأس القائمة المزاراة، نستخدم (!, fail) لقطع البحث فوراً وإخبار البرنامج بأن يتوقف لانه زرنا هذا المطار سابقاً في هذه الرحلة حتى لا نقوم بزيارته مجدداً كي لا ندخل في حلقة لانهاية

شروط التوقف:

`! :- path(Y, Y, _, Visited, Visited)`

عندما يصبح المطار الحالي هو نفسه مطار الهدف Y فهذا يعني أننا وصلنا بنجاح ونقوم بنسخ القائمة المزاراة كاملة إلى متغير النتيجة ونضع (!) ليتوقف البرنامج ولا يبحث عن مسارات أطول بلا داع.

الحالة العودية (الانتقال عبر مطار وسيط):

```
path(X, Y, N, Visited, Route) :-
    N > 0,
    connected(X, Z),
    \+ has_visited(Z, Visited),
    N1 is N - 1,
    path(Z, Y, N1, [Z|Visited], Route).
```

لماذا وضعنا $N > 0$ و $N1 \text{ is } N - 1$ ؟ N هي بمثابة عداد وحددنا له في الدالة الأساسية القيمة (5) أي في كل مطار نمر به ينقص العداد بمقدار 1 فإذا دار البرنامج 5 مرات ولم يصل، يتوقف فوراً أي من أجل منع اللوب اللانهائية لماذا وضعنا $\backslash + \text{has_visited}$ ؟ إشارة $\backslash +$ تعني النفي (NOT) نحن نشتط على البرنامج ان يبحث عن مطار مجاور Z ولكن بشرط ألا يكون Z قد زُرناه من قبل ($\backslash + \text{has_visited}$) لماذا وضعنا $[Z|Visited]$ ؟ عندما يقرر البرنامج الذهاب للمطار الجديد Z يقوم بإضافته فوراً إلى رأس قائمة المطارات المزارة

↪ الدالة الرئيسية وعكس النتيجة (route):

```
route(X, Y, Route) :-
    path(X, Y, 5, [X], ReverseRoute),
    reverse(ReverseRoute, Route).
```

عند استدعاء التابع نبدأ الرحلة من المطار X لذلك نمرر قائمة الزيارة الابتدائية تحتوي عليه فقط $[X]$ لماذا استخدمنا reverse ؟ لأن البرولوج في الحالة العودية يقوم بإضافة المطارات الجديدة إلى رأس القائمة فإذا سافرنا من لندن ثم باريس ثم روما، ستخزن في القائمة كـ $[roma, paris, london]$ لذلك نقوم بعكسها في النهاية باستخدام التابع الجاهز reverse لتعود للترتيب الصحيح المقروء.
فحص إمكانية السفر فقط (travelFromTo) :

`!, travelFromTo(X, Y) :- route(X, Y, _)`

الطلب الثاني في السؤال كان يريد دالة تخبرنا فقط بـ true أو false إن كان هناك خط سفر أم لا (دون طباعة القائمة) لذلك قمنا باستدعاء `route(X, Y, _)` ووضعنا شرط المتغير المجهول `_` لأننا لا نهتم بشكل المسار ووضعنا (!) لينتهي الأمر بمجرد العثور على أول طريق متاح.

○ الاستدعاء للبحث عن المسار الكامل من (athens <-- (London)

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
?-
% c:/Users/Dell/Desktop/telegram downloads/prolog/main.pl compiled 0.00
sec, 14 clauses
?-
| route(london, athens, Route).
Route = [london, paris, rome, athens]
```

○ استدعاء للتحقق من إمكانية السفر من مدينة الى الأخرى:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
?- travelFromTo(madrid, lisbon).
true.
?-
```

○ استدعاء لمحاولة البحث عن مسار من مدينة الى أخرى:

```
SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
?- route(madrid, london, Route).
false.
?-
```

ملاحظة:

في برولوج يمكننا استخدام Template أو غلاف نختاره لتغيير شكل النتيجة النهائية داخل القائمة أي بدلاً من أن يعطينا العناصر مجردة يقوم بأخذ كل عنصر ووضعه داخل الكلمة التي حددناها له لنفترض أن لدينا أسماء الطلاب الناجحين ahmed و nour : `findall(X, student(X), R).` يعطينا الأسماء مجردة كما هي: `R = [ahmed, nour]` الاستدعاء باستخدام القالب `happy(X)`: `findall(happy(X), student(X), R)` يأخذ كل اسم ويغلفه بكلمة `happy`: `R = [happy(ahmed), happy(nour)]` أي ان الفائدة من استخدام Template:

تغيير شكل النتيجة (قالب): بدل أن يعطينا البرنامج الأسماء مجردة نحن من يحدد شكلها النهائي مثل اختيار أن تظهر النتيجة كـ `happy(charlotte)` بدلاً من `charlotte` تجهيز البيانات لتابع آخر: عندما نريد تمرير هذه القائمة المجمعة إلى تابع آخر في الكود يتوقع استقبال عناصر مغلفة بكلمة `happy` ليفحصها أو يعالجها فنكون اختصرنا هذه الخطوة مسبقاً

✎ خوارزمية ترتيب العناصر بالدرج (Insertion Sort):

هذه الخوارزمية تعتمد على تقسيم المشكلة إلى جزأين أساسيين، وهما دالة الترتيب (Sort) التي تمر على عناصر القائمة، ودالة الإدراج (Insert) التي تضع كل عنصر في مكانه الصحيح داخل قائمة فرعية مرتبة

مثال: لدينا قائمة (List) نريد ترتيبها وذلك من خلال إنشاء قائمة جديدة فارغة يتم ملؤها بعناصر القائمة الأولى واحداً تلو الآخر مع وضع كل عنصر في مكانه المناسب



```

main.pl
File Edit Browse Compile Prolog Pce Help
main.pl
insert(X, [], [X]).
insert(X, [H|T], [X, H|T]) :- X <= H, !.
insert(X, [H|T], [H|NT]) :- insert(X, T, NT).
my_sort([], []).
my_sort([H|T], Result) :-
my_sort(T, Partial, insert(H, Partial, Result)).

```

٤ تم تقسيم الحل إلى تابعين أساسيين لتسهيل التفكير في المشكلة:

- **تابع الترتيب (my_sort):** وظيفته المرور على عناصر القائمة الأصلية واستدعاء تابع الإدراج يأخذ مصفوفة (قائمة) ويرجع مصفوفة مرتبة ويعمل بشكل عودي حيث يقوم بترتيب (Tail) القائمة أولاً ثم يقوم بإدراج (Head) القائمة في تلك القائمة المرتبة باستخدام تابع الإدراج وبمجرد أن يصبح لديه جزء مرتب، يرسله لتابع الإدراج ليضع فيه العنصر الحالي في مكانه الصحيح
- **تابع الإدراج (insert):** وظيفته إضافة العنصر في مكانه الصحيح داخل القائمة التي تم ترتيب أجزائها بالفعل ويأخذ هذا التابع ثلاثة بارامترات: العنصر المراد إدراجه، القائمة الجزئية المرتبة، والنتيجة النهائية بعد الإدراج وله عدة حالات منطقية:

1. حالة القائمة الفارغة (Base Case): إذا كانت القائمة التي سنضيف إليها فارغة، يتم ببساطة وضع العنصر في القائمة وتنتهي العملية.
2. حالة العنصر أصغر من أو يساوي Head ($X \leq \text{Head}$): إذا كان العنصر المراد إضافته أصغر من أول عنصر في القائمة الحالية، يُوضع العنصر في البداية (قبل Head)، وتتوقف العودية هنا لأنه تم العثور على المكان المناسب
3. حالة العنصر أكبر من Head ($X > \text{Head}$): في حال لم يتحقق الشرط السابق، يتم الاحتفاظ بال Head الحالي كما هو ويتم استدعاء تابع الإدراج بشكل عودي لإضافة العنصر في (Tail) القائمة

٤ دعونا نتبع هذا الاستدعاء للقائمة التالية:

- يتم مقارنة 3 مع العنصر الأول (2)؛ بما أن 3 أكبر، ننتقل للعنصر التالي.
- يتم مقارنة 3 مع العنصر الثاني (1)؛ بما أن 3 أكبر، ننتقل للعنصر التالي.
- يتم مقارنة 3 مع العنصر الثالث (4)؛ بما أن 3 أصغر، يتم وضعه قبل الرقم 4.
- نأخذ النتائج عودياً أي من الأسفل للأعلى لتعود القائمة كاملة ومرتبّة

```

SWI-Prolog (AMD64, Multi-threaded, version 8.4.2)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 8.4.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?-
% c:/Users/Dell/Desktop/telegram downloads/prolog/main.pl compiled 0.00
sec, 5 clauses
?-
| my_sort([3, 4, 1, 2], Result).
Result = [1, 2, 3, 4]

```